```java
import java.math.BigInteger;
import java.util.Scanner;

public class rsa10{

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // Step 1: Select two prime numbers
        BigInteger p = BigInteger.valueOf(61);
        BigInteger q = BigInteger.valueOf(53);

        // Step 2: Calculate n = p × q
        BigInteger n = p.multiply(q);

        // Step 3: Calculate ϕ(n) = (p - 1)(q - 1)
        BigInteger phi =
p.subtract(BigInteger.ONE).multiply(q.subtract(BigInteger.ONE));

        // Step 4: Select public exponent e
        BigInteger e = BigInteger.valueOf(17);

        // Step 5: Calculate private exponent d
        BigInteger d = e.modInverse(phi);

        System.out.println("RSA Key Generation");
        System.out.println("------------------");
        System.out.println("Prime number p = " + p);
        System.out.println("Prime number q = " + q);
        System.out.println("n = p * q = " + n);
        System.out.println("phi(n) = " + phi);

        System.out.println("\nPublic Key  = (" + e + ", " + n +
")");
        System.out.println("Private Key = (" + d + ", " + n + ")");

        System.out.print("\nEnter message as a number less than "+ n
+ ": ");

        BigInteger message = sc.nextBigInteger();

        if (message.compareTo(BigInteger.ZERO) < 0 ||
            message.compareTo(n) >= 0) {

            System.out.println("Message must be between 0 and "
+n.subtract(BigInteger.ONE));
```

```
            sc.close();
            return;
        }

        // Encryption: C = M^e mod n
        BigInteger encrypted = message.modPow(e, n);

        // Decryption: M = C^d mod n
        BigInteger decrypted = encrypted.modPow(d, n);

        System.out.println("\nOriginal message  : " + message);
        System.out.println("Encrypted message : " + encrypted);
        System.out.println("Decrypted message : " + decrypted);

        sc.close();
    }
}
```